

U.T. 6. Utilización del modelo de objetos del documento (DOM).

DOM o *Document Object Model* es un conjunto de utilidades específicamente diseñadas para manipular documentos XML. Por extensión, DOM también se puede utilizar para manipular documentos XHTML y HTML. Técnicamente, DOM es una API de funciones que se pueden utilizar para manipular las páginas XHTML de forma rápida y eficiente.

2.1. EL MODELO DE OBJETOS DEL DOCUMENTO (DOM)

Es un estándar W3C que define cómo acceder a los documentos como por ejemplo xml y html.

Es una interfaz de programación de aplicaciones (API) de la plataforma de W3C.

Permite a los scripts acceder y actualizar dinámicamente su contenido, estructura y estilo de documento.

2.1.1. Tipos de modelos DOM

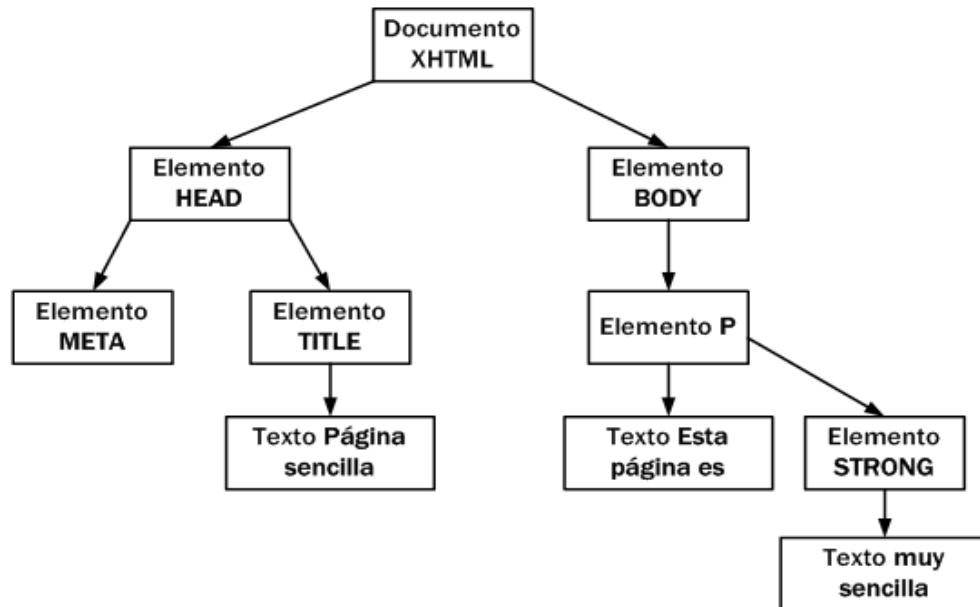
El núcleo del DOM es el modelo estándar para cualquier documento estructurado. Define especificaciones a nivel general de pautas para definición de objetos, propiedades del documento y como acceder a ellos.

XML DOM es el modelo estándar para documentos XML. Define los objetos y propiedades de los elementos XML y como acceder a ellos.

HTML DOM es el modelo estándar para documentos HTML. Define una interfaz de programación estándar independiente de la plataforma y el lenguaje. Está definido en la W3C. El objeto DOM HTML es un estándar sobre la forma de obtener, modificar, añadir o eliminar elementos HTML.

2.1.2. Estructura del árbol DOM

Aunque en ocasiones DOM se asocia con la programación web y con JavaScript, la API de DOM es independiente de cualquier lenguaje de programación. De hecho, DOM está disponible en la mayoría de lenguajes de programación comúnmente empleados.



Aquí tenemos un esquema sencillo de la representación de la estructura del Árbol DOM.

```
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<meta http-equiv="Content-Type" content="text/html; charset=iso-8859-
1" />
<title>Página sencilla</title>
</head>
<body>
<p>Esta página es <strong>muy sencilla</strong></p>
</body>
</html>
```

Para ordenar la estructura del árbol, existe una serie de reglas:

- En el árbol de nodos, al nodo superior (document), se le llama raíz.
- Cada nodo, exceptuando el nodo raíz, tiene un padre.
- Un nodo puede tener cualquier número de hijos.
- Una hoja es un nodo sin hijos.
- Los nodos que comparten el mismo padre, son hermanos.

2.2. OBJETOS DEL MODELO. PROPIEDADES Y METODOS.

Una vez creado el árbol DOM, se obtienen sus nodos. Que son cada una de las partes del árbol. Cada propiedad es *nodeType*. Existen 12 tipos de nodos disponibles y cada uno tiene sus propiedades asociadas.

2.2.1. Objetos del modelo

Según la W3C, éstos son los tipos de nodos más importantes:

- **Document:** Es el nodo raíz del documento HTML, todos los documentos del árbol cuelgan de él.
- **DocumentType:** Este nodo indica la representación del DTD de la página. Un DTD es una definición de tipo de documento. Define la estructura y sintaxis de un documento XML. El DOCTYPE es el encargado de indicar el DocumentType.
- **Element:** Este nodo representa el contenido de una pareja de etiquetas de apertura y cierre (<etiqueta>...</etiqueta>). También puede representar una etiqueta abreviada que se cierra a si misma (
). Este es el único nodo que puede tener tantos nodos hijos como atributos.
- **Attr:** Este nodo representa el nombre del atributo o valor.
- **Text:** Este nodo almacena la información que es contenida en el tipo de nodo Element.

- **CDataSection:** Este nodo representa una secuencia de código del tipo `<![CDATA[ ]]>`: Este texto solo será analizado por un programa de análisis.
- **Comment:** Este nodo representa un comentario XML.

2.2.2. La interfaz Node

Para poder manipular la interfaz de los nodos, JavaScript crea un objeto llamado Node.

Este objeto define una serie de constantes que identifican los tipos de nodo.

- Node.ELEMENT_NODE = 1
- Node.ATTRIBUTE_NODE = 2
- Node.TEXT_NODE = 3
- Node.CDATA_SECTION_NODE = 4
- Node.ENTITY_REFERENCE_NODE = 5
- Node.ENTITY_NODE = 6
- Node.PROCESSING_INSTRUCTION_NODE = 7
- Node.COMMENT_NODE = 8
- Node.DOCUMENT_NODE = 9
- Node.DOCUMENT_TYPE_NODE = 10
- Node.DOCUMENT_FRAGMENT_NODE = 11
- Node.NOTATION_NODE = 12

Además de estas constantes, Node proporciona las siguientes propiedades y métodos:

Propiedad/Método	Valor devuelto	Descripción
nodeName	String	El nombre del nodo (no está definido para algunos tipos de nodo)
nodeValue	String	El valor del nodo (no está definido

		para algunos tipos de nodo)
nodeType	Number	Una de las 12 constantes definidas anteriormente
ownerDocument	Document	Referencia del documento al que pertenece el nodo
firstChild	Node	Referencia del primer nodo de la lista childNodes
lastChild	Node	Referencia del último nodo de la lista childNodes
childNodes	NodeList	Lista de todos los nodos hijo del nodo actual
previousSibling	Node	Referencia del nodo hermano anterior o null si este nodo es el primer hermano
nextSibling	Node	Referencia del nodo hermano siguiente o null si este nodo es el último hermano
hasChildNodes()	Boolean	Devuelve true si el nodo actual tiene uno o más nodos hijo
attributes	NamedNodeMap	Se emplea con nodos de tipo Element. Contiene objetos de tipo Attr que definen todos los atributos del elemento
appendChild(nodo)	Node	Añade un nuevo nodo al final de la lista childNodes
removeChild(nodo)	Node	Elimina un nodo de la lista childNodes
replaceChild(nuevoNodo, anteriorNodo)	Node	Reemplaza el nodo anteriorNodo por el nodo nuevoNodo
insertBefore(nuevoNodo, anteriorNodo)	Node	Inserta el nodo nuevoNodo antes que la posición del nodo anteriorNodo dentro de la lista childNodes

Los métodos y propiedades incluidas en la tabla anterior son específicos de XML, aunque pueden aplicarse a todos los lenguajes basados en XML, como por ejemplo XHTML. Para las páginas creadas con HTML, los navegadores hacen como si HTML estuviera basado en XML y lo tratan de la misma forma. No obstante, se han definido algunas extensiones y particularidades específicas para XHTML y HTML.

2.3. ACCESO AL DOCUMENTO DESDE CÓDIGO

Cuando el árbol de nodos DOM ha sido construido por el navegador de forma automática, podemos acceder a cualquier nodo. Si existe más de un elemento, estos se van almacenando en un array. Por ejemplo:

```
<html>
<head>
<title>TituloDOM</title>
</head>
<body>
<p>ParrafoDOM</p>
<p>ParrafoDOM segundo</p>
<p>ParrafoDOM tres</p>
</body>
</html>
```

El primer paso sería recoger los datos del objeto que representa el elemento raíz de la página:

```
var obj_html= document.documentElement;
```

Después, accederíamos al primer y último hijo del nodo:

```
var obj_head= obj_html.firstChild;
var obj_body= obj_html.lastChild;
```

En caso de haber más nodos, accederíamos a ellos desde el índice:

```
var obj_head= obj_html.childNodes[0];
var obj_body= obj_html.childNodes[1];
```

Si no sabemos el número de hijos, podemos acceder a este valor usando la propiedad “.length”.

```
var numeroHijos= obj_html.childNodes.length;
```

2.3.1. Acceso a los tipos de nodo

Los objetos del modelo se diferencian por su tipo:

Existen diferentes tipos de Nodo y su forma de acceder es la siguiente:

```
obj_tipo_document= document.nodeType; //9  
obj_tipo_elemento= document.documentElement.nodeType; //1
```

En el primer caso, el valor 9 es igual al definido en la constante Node.DOCUMENT_NODE. En el segundo ejemplo, el valor 1 coincide con la constante Node.ELEMENT_NODE.

Afortunadamente, no es necesario memorizar los valores numéricos de los tipos de nodos, ya que se pueden emplear las constantes predefinidas:

```
alert(document.nodeType == Node.DOCUMENT_NODE); // true  
alert(document.documentElement.nodeType == Node.ELEMENT_NODE); // true
```

El único navegador que no soporta las constantes predefinidas es Internet Explorer 7 y sus versiones anteriores, por lo que si se quieren utilizar es necesario definir las de forma explícita:

2.3.2. Acceso directo a los nodos

Para acceder de manera directa a los distintos nodos, existen los siguientes métodos:

- `getElementsByTagName();`
- `getElementsByName();`
- `getElementById();`

`getElementsByTagName()`: Este método recupera los elementos de la página HTML que le hayamos pasado por parámetro.

```
var divs= document.getElementsByTagName("div");
```

getElementsByTagName(): Este método recupera los datos de la página HTML donde “*name*” coincide con el name que le hayamos pasado en la función. Un ejemplo es el siguiente:

```
var divPrimero= document.getElementsByTagName("primero");  
<div name="primero">...</div>  
<div name="segundo">...</div>  
<div>...</div>
```

getElementById: Este método recupera el elemento HTML cuyo ID coincida con el nombre pasado en la función. Ejemplo:

```
var elemento1 = document.getElementById("elemento1");  
<div id="pie">  
    <a href="URL" id="imagen">...</a>  
</div>
```

2.3.3. Acceso a los atributos de un nodo tipo element

DOM permite acceder directamente a todos los atributos de una etiqueta.

La propiedad `attributes` permite acceder a los atributos de un nodo de tipo `element` mediante los siguientes métodos:

getNamedItem(nomAttr): Devuelve el nodo de tipo `attr` (atributo), cuya propiedad `nodeName` (Nombre del nodo) contenga el valor `nomAttr`.

removeNamedItem(nomAttr): Elimina el nodo de tipo `attr` (atributo) en el que la propiedad `nodeName` (Nombre del nodo) coincida con el valor `nomAttr`.

setNamedItem(nodo). Este método añade el nodo `attr` (atributo) a la lista de atributos del nodo `element`. Lo indexa según la propiedad `nodeName` (del atributo).

item(pos). Devuelve el nodo correspondiente a la posición indicada por el valor numérico `pos`.

DOM proporciona algunos métodos que permiten un acceso directo a la modificación, inserción y borrado de los atributos de una etiqueta:

getAttribute(nomAtributo): Este método equivale a:
`attributes.getItem(nombAtributo).`

setAttribute(nomAtributo, valorAtributo). Este método equivale a la estructura: `attributes.getItem(nomAtributo).value= valor`

removeAttribute(nomAtributo). Este método equivale a la estructura: `attributes.removeItem(nomAtributo)`

2.3.4. Creación y eliminación de nodos

Para generar nodos de forma dinámica existen los siguientes métodos:

createAttribute(nomAtributo): Este método crea un nodo de tipo atributo con el nombre pasado a la función.

createCDATASection(textoPasado): Este método crea una sección de tipo CDATA con un nodo hijo de tipo texto con el valor textoPasado.

createComment(textoPasado): Este método crea un nodo de tipo comment(comentario), con el contenido de textoPasado.

createDocumentFragment(): Este método crea un nodo de tipo DocumentFragment.

createElement(nomEtiqueta): Este método crea un elemento del tipo etiqueta, del tipo del parámetro pasado como nomEtiqueta.

createEntityReference(nomNodo): Este método crea un nodo de tipo EntityReference.

createProcessingInstruction(objetivo,dato): Este método crea un nodo de tipo ProcessingInstruction.

createTextNode(textoPasado): Este método crea un nodo de tipo texto con el valor del parámetro pasado, textoPasado.

2.4. PROGRAMACIÓN DE EVENTOS

Se utilizan para relacionar la interacción del usuario con las acciones de DOM.

2.4.1. Carga de la página HTML

Una condición para que se genere la estructura de árbol es que la página se cargue completamente. Por este motivo es necesario conocer si se ha cargado y para ello se utiliza el evento onload.

```
<html>
  <head><title>TituloDOM</title></head>
  <body onload="alert('Página cargada completamente');">
    <p>Primer parrafo</p>
  </body>
</html>
```

2.4.2. Comprobar si el árbol DOM está cargado

```
<html>
  <head><title>TituloDOM</title>
  <script>
    function cargada() {
      window.onload= "true";
      if(window.onload){ return true; }
      return false;
    }
    function pulsar(){
      if(cargada()){ alert("Página cargada correctamente");}
    }
  </script>
</head>
  <body> <p onclick="pulsar();">Primer párrafo</p> </body>
</html>
```

2.4.3. Actuar sobre el DOM al desencadenarse eventos

```
<html>
  <head><title>TituloDOM</title>
```

```
<script>
function ratonEncima() {
    document.getElementsByTagName("div")[0].childNodes[0].nodeValue =
"EL RATON ESTA ENCIMA";
}
function ratonFuera() {
    document.getElementsByTagName("div")[0].childNodes[0].nodeValue =
"NO ESTA EL RATON ENCIMA";
}
</script>
</head>
<body>
    <div onmouseover="ratonEncima(); "onmouseout="ratonFuera();">
        VALOR POR DEFECTO
    </div>
</body>
</html>
```

2.5. DIFERENCIAS EN LAS IMPLEMENTACIONES DEL MODELO

Una de las principales dificultades a la hora de utilizar DOM es que no todos los navegadores hacen una misma interpretación.

La guerra entre navegadores a la hora de generar sus propios estándares, ha generado muchos problemas a los programadores de páginas web.

2.5.1. Adaptaciones de código para diferentes navegadores

Pueden ocasionar problemas de interpretación. Complican la actualización futura de la página. Generan la necesidad de modificar el código implementado.

Internet Explorer ha añadido su propia extensión de DOM, con lo cual genera problemas de interoperabilidad entre los navegadores web

Para mejorar la compatibilidad, podemos crear de forma explícita las constantes predefinidas.

```
alert(Node.DOCUMENT_NODE); // Devolvería 9
alert(Node.ELEMENT_NODE); // Devolvería 1
alert(Node.ATTRIBUTE_NODE); // Devolvería 2
```

```
if(typeofNode == "undefined") {
    varNode = {
        ELEMENT_NODE: 1,
        ATTRIBUTE_NODE: 2,
        TEXT_NODE: 3,
        CDATA_SECTION_NODE: 4,
        ENTITY_REFERENCE_NODE: 5,
        ENTITY_NODE: 6,
        PROCESSING_INSTRUCTION_NODE: 7,
        COMMENT_NODE: 8,
        DOCUMENT_NODE: 9,
        DOCUMENT_TYPE_NODE: 10,
        DOCUMENT_FRAGMENT_NODE: 11,
        NOTATION_NODE: 12
    };
}
```

2.6. USO DE LIBRERÍAS DE TERCEROS

Para solucionar el problema del desarrollo de aplicaciones web en diferentes navegadores nace **cross-browser**. Tiene la intención de visualizar una página o aplicación web exactamente igual en todos los navegadores. Para conseguir este objetivo surgen utilidades que permiten unificar los eventos y sus propiedades.

Renderizar a través de una Web:

Existen páginas Web que nos permiten introducir una dirección de una página Web y elegir la versión del navegador con el que queremos visualizarlo.

Por ejemplo, **netrenderer** nos permite visualizar una página Web en las distintas versiones de Internet Explorer.

Normalmente este tipo de páginas, solo nos muestran una imagen del resultado, por lo que puede resultar complicado en ocasiones solucionar algunos problemas.

Programas para renderizar:

Existen programas que nos permiten instalar varias versiones del mismo navegador, como multiIE o Internet Explorer Collection. Estos programas dan problemas de compatibilidad de versiones con los últimos navegadores.

Instalar los navegadores en máquinas virtuales.

Otra opción es instalar las versiones de navegadores en máquinas virtuales, que además estén acorde con los sistemas operativos para los que hay instalables de la versión de navegador.

Para el uso de las utilidades *cross-browser* es necesario implementar funciones que habitualmente vienen definidas en librerías.

Es necesario comprobar el navegador y tomar una decisión u otra en función de la respuesta.

```
if (window.XMLHttpRequest){
    // codigoparaIE7+, Firefox, Chrome, Opera, Safari
    Xmlhttp= new XMLHttpRequest();
}else{
    // codigoparaIE6, IE5
    Xmlhttp= new ActiveXObject("Microsoft.XMLHTTP");
}
```